

Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ciencias y Sistemas
Organización de Lenguajes y Compiladores 1

MANUAL TECNICO

Dionicio Dario Simon Bal

Carnet: 201905487

Guatemala 23 de Octubre del 2024

Requerimientos mínimos del sistema

Para el uso correcto de la aplicación es necesario los siguientes componentes y programas:

- Memoria RAM 2GB o superior
- Espacio disponible en disco de 500 MB
- Angular versión 11.2.9
- Typescript 4.2.4 o superior
- NodeJS 6.14.7
- Graphviz versión 2.38.0 (20140413.2041)
- Sistema Operativo Windows 10 x64

Paradigma utilizado en el proyecto

Para la creación de la aplicación se utilizó un paradigma orientado a objetos debido a que por el tipo de funcionamiento que se requería era el más óptimo para la realización de la aplicación, las clases utilizadas en la aplicación fueron necesarias para crear contenedores de datos específicos para su manejo de una manera más sencilla y optima, todos los árboles, expresiones regulares y conjuntos fueron registrados en nodos con sus respectivos datos y almacenados en listas de estos mismos tipos. Para la creación de los árboles se utilizó una metodología ascendente realizando todo el proceso inicialmente desde sus hojas hasta llegar a la raíz.

Explicación del lenguaje a utilizar

La aplicación como es un intérprete puede realizar distintas operaciones aritméticas, operaciones lógicas, ciclos, funciones, declaración y llamado de variables, métodos, todo programa para poder funcionar deberá de tener una función run la cual es la función principal de la ejecución de la aplicación, se explicará la estructura de cada cosa.

1. Operaciones aritméticas: Las operaciones aritméticas que se pueden realizar son +, -, *, /, ^, % y todas poseen la misma estructura la cual es "EXP OPERACIÓN EXP" siendo exp cualquier valor u otra operación posible, también existe la operación de negación la cual si cambia su estructura por "MENOS EXP" es la única que utiliza esta estructura.

2. Operaciones Lógicas: Las operaciones lógicas que puede ser utilizadas son ==, !=, &, las cuales poseen la siguiente estructura "EXP LOGICA EXP" exceptuando el NOT la cual posee la estructura "NOT EXP".

3. Ciclos

a. FOR: la estructura de un for es la siguiente "FOR (DECLARACION / ASIGNACION ; CONDICION ; ACTUALIZACION) { INSTRUCCIONES }"

b. WHILE: la estructura de un while es la siguiente "WHILE(CONDICION){INSTRUCCIONES}"

c. DO WHILE: la estructura de un while es la siguiente "DO {INSTRUCCIONES} WHILE(CONDICION)"

4. Nativas: existen diferentes funciones nativas con la estructura "FUNCION(EXP)" en las cuales podemos mencionar, toString, length, round, truncate, toUpper, toLower

5. Funciones y métodos

a. Funciones: los métodos tienen la siguiente estructura "TIPO ID (PARAMETROS){ INSTRUCCIONES }"

b. Métodos: los métodos tienen la siguiente estructura "ID (PARAMETROS) VOID { INSTRUCCIONES } "

6. Run: run es una forma de llamar una función y a la vez indicara cual es nuestra función principal a ejecutar, solo puede existir uno por terminal "EXEC ID (PARAMETROS) "

7. LLAMADAS: para llamar una función o un método se utiliza la siguiente estructura " ID (PARAMETROS) "

Para explicar de mejor manera esta estructura vamos a ver la gramática, la cual fue generada con la aplicación JISON primero explicaremos las expresiones y símbolos que se podrán utilizar:

Caracteres que se ignoran

```
/* Espacios en blanco */
"//".*      {}
[/][*][^]*[*]+([/^][^]*[*]+)*[/]      {}
[ \r\t]+    {}
\n          {}
\s+         {}
[']         {Texto=""; this.begin("CARACTER");}
```

Caracteres y Cadenas

Para estos dos casos fue utilizados los CASOS de jison los cuales nos permiten cambiar de expresión a utilizar dependiendo de la entrada que tiene la expresión.

- Carácter

```
<CARACTER>[^'\\]"    {yytext = yytext.substr(0,yytext.length-1); this.popState(); return 'CARACTER';}
<CARACTER>"\\n"      {yytext = '\n'; this.popState(); return 'CARACTER';}
<CARACTER>"\\t"      {yytext = '\t'; this.popState(); return 'CARACTER';}
<CARACTER>"\\r"      {yytext = '\r'; this.popState(); return 'CARACTER';}
<CARACTER>"\\\"      {yytext = '\"'; this.popState(); return 'CARACTER';}
<CARACTER>"\\'      {yytext = '\''; this.popState(); return 'CARACTER';}
<CARACTER>"\\\\      {yytext = '\\'; this.popState(); return 'CARACTER';}
<CARACTER><<EOF>>    return "EOF_IN_CHARACTER";
<CARACTER>[^'\\]"    {this.popState(); return 'CARACTER_ERROR';}
```

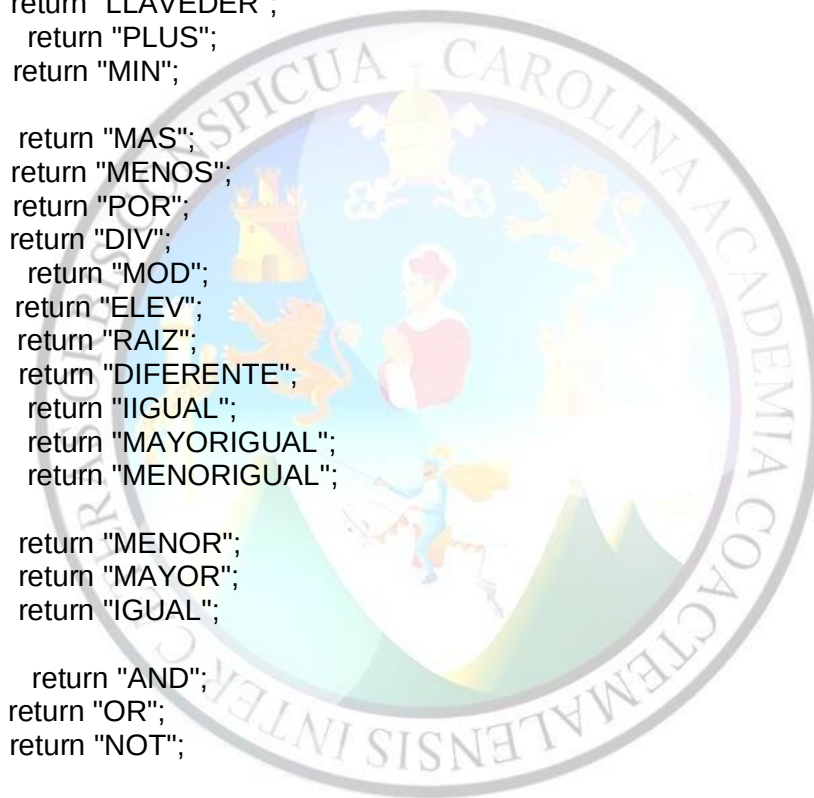
- Cadena

```
["]              {Texto=""; this.begin("Cadena");}
<Cadena>[^"\\]+  {Texto+=yytext;}
<Cadena>"\\n"     {Texto+='\n';}
<Cadena>"\\t"     {Texto+='\t';}
<Cadena>"\\r"     {Texto+='\r';}
<Cadena>"\\\"      {Texto+= '\"';}
<Cadena>"\\'      {Texto+= '\'';}
<Cadena>"\\\\      {Texto+= '\\';}
<Cadena><<EOF>>    return "EOF_IN_STRING";
<Cadena>["]      {yytext = Texto; this.popState(); return 'Cadena';}
```

Símbolos

Todos estos son los símbolos que podemos utilizar

"echo"	return "PRINT";
","	return "PTCOMA";
":"	return "DOSPT"
."	return "PT";
","	return "COMA";
"("	return "PARIZ";
")"	return "PARDER";
"["	return "CORIZ";
"]"	return "CORDER";
"{"	return "LLAVEIZ";
"}"	return "LLAVEDER";
"++"	return "PLUS";
"--"	return "MIN";
"+"	return "MAS";
"_"	return "MENOS";
"*"	return "POR";
"/"	return "DIV";
"%"	return "MOD";
"^"	return "ELEV";
"\$"	return "RAIZ";
"!="	return "DIFERENTE";
"=="	return "IIGUAL";
">="	return "MAYORIGUAL";
"<="	return "MENORIGUAL";
"<"	return "MENOR";
">"	return "MAYOR";
"="	return "IGUAL";
"&&"	return "AND";
" "	return "OR";
"!"	return "NOT";
"int"	return "INT";
"double"	return "DOUBLE";
"bool"	return "BOOLEAN";
"char"	return "CHAR";
"string"	return "STRING";
"true"	return "TRUE";
"false"	return "FALSE";
"let"	return "LET";
"const"	return "CONST";
"vector"	return "VECTOR";



"FOR"	return "FOR";
"UNTIL"	return "UNTIL";
"DO"	return "DO";
"SWITCH"	return "SWITCH";
"IF"	return "IF";
"ELSE"	return "ELSE";
"LOOP"	return "LOOP";
"WHILE"	return "WHILE";
"NEW"	return "NEW";
"RETURN"	return "RETURN";
"CONTINUE"	return "CONTINUE";
"BREAK"	return "BREAK";
"LIST"	return "LIST";
"ADD"	return "ADD";
"CASE"	return "CASE";
"DEFAULT"	return "DEFAULT";
"VOID"	return "VOID";
"LOWER"	return "LOWER";
"UPPER"	return "UPPER";
"Len"	return "LENGTH";
"truncate"	return "TRUNCATE";
"round"	return "ROUND";
"Typeof"	return "TYPEOF";
"toString"	return "TOSTRING";
"toCharArray"	return "CHARARRAY";
"EJECUTAR"	return "EJECUTAR";
"CAST"	return "CAST";
"AS"	return "AS";
"IS"	return "IS";
"REVERSE"	return "REVERSE";
"MAX"	return "MAX";
"MIN"	return "MIN2";
"SUM"	return "SUM";
"AVERAGE"	return "AVERAGE";
"FUNCTION"	return "FUNCTION";

Analizador Sintáctico

Se muestra el código utilizado para el analizador sintáctico, para la explicación de cada cosa ver el documento de gramática.

/* Asociación de operadores y precedencia */

%left 'TERNARIO'

%left 'OR'

%left 'AND'

%right 'NOT'

%left 'IIGUAL' 'DIFERENTE' 'MENOR' 'MENORIGUAL' 'MAYOR' 'MAYORIGUAL'

```
%left 'MAS' 'MENOS'
%left 'POR' 'DIV' 'MOD'
%left 'ELEV'
%right UMENOS
%right FCAST
```

```
[A-Za-z]([A-Za-z][0-9][_])* return "ID";
[0-9]+ "." [0-9]+ \b      return "DOBLE";
[0-9]+ \b                return "ENTERO";
```

```
<<EOF>>      return 'EOF';
```

```
. {
    errores.push(new Error("Error Lexico", `Error Lexico, caracter '${yytext}' no esperado.` ,
yyloc.first_line, yyloc.first_column));
```

```
    return 'ERROR_LEXICO';
}
/lex
```

```
/* Asociación de operadores y precedencia */
```

```
%left 'PARDER'
%left 'DOSPT'
%left 'OR'
%left 'AND'
%right 'NOT' 'IS'
%left 'IIGUAL' 'DIFERENTE' 'MENOR' 'MENORIGUAL' 'MAYOR' 'MAYORIGUAL'
%left 'MAS' 'MENOS'
%left 'POR' 'DIV' 'MOD'
%left 'ELEV' 'RAIZ'
%right UMENOS
%right FCAST
%left 'PLUS','MIN'
```

```
%start INIT
```

```
%%
```

```
INIT
: LINS EOF    {return $1;}
;
```

LINS

```
:LINS INS    {$1.push($2); $$=$1;}  
|INS        {$$= [$1];}
```

;

INS

```
: IMPRIMIR PTCOMA { $$ = $1; }  
| FEJECUTAR PTCOMA { $$ = $1; }  
| DECLARACION PTCOMA { $$ = $1; }  
| ASIGNACION PTCOMA { $$ = $1; }  
| DTERNARIO PTCOMA { $$ = $1; }  
| FFOR { $$ = $1; }  
| FWHILE { $$ = $1; }  
| INCREMENTO PTCOMA { $$ = $1; }  
| DECREMENTO PTCOMA { $$ = $1; }  
| FIF { $$ = $1; }  
| FSWITCH { $$ = $1; }  
| FBREAK { $$ = $1; }  
| FCONTINUE { $$ = $1; }  
| DOWHILE PTCOMA { $$ = $1; }  
| FLOOP { $$ = $1; }  
| FRETURN { $$ = $1; }  
| FFUNCION { $$ = $1; }  
| LLAMADA PTCOMA { $$ = $1; }
```

```
|error { errores.push(new Error("Error Sintactico", `Error Sintactico, caracter '${yytext}' no  
esperado.` , this._$.first_line, this._$.first_column));
```

```
    console.error('Este es un error sintáctico: ' + yytext + ', en la linea: ' + this._$.first_line + ', en la  
columna: ' + this._$.first_column); }
```

;

FEJECUTAR

```
:EJECUTAR LLAMADA { $$ = new Ejecutar($2, this._$.first_line, this._$.first_column); }
```

;

DECLARACION

```
: LET LID DOSPT TIPODATO { $$ = new DecVariable($1, $4, $2, null,this._$.first_line,  
this._$.first_column); }
```

```
| LET LID DOSPT TIPODATO IGUAL EXP { $$ = new DecVariable($1, $4, $2,  
$6,this._$.first_line, this._$.first_column); }
```

```
| CONST LID DOSPT TIPODATO IGUAL EXP { $$ = new DecVariable($1, $4, $2,  
$6,this._$.first_line, this._$.first_column); }
```

```
| CONST LID DOSPT TIPODATO { $$ = new DecVariable($1, $4, $2, null,this._$.first_line,  
this._$.first_column); }
```

```
| LET LID DOSPT TIPODATO CORIZ CORDER IGUAL NEW VECTOR TIPODATO CORIZ EXP  
CORDER { $$ = new DecVectores($2, $4,this._$.first_line, this._$.first_column, $12); }
```

```
| LET LID DOSPT TIPODATO CORIZ CORDER CORIZ CORDER IGUAL NEW VECTOR  
TIPODATO CORIZ EXP CORDER CORIZ EXP CORDER { $$ = new DecVectores($2, $4,  
this._$.first_line, this._$.first_column, $14, $17); }
```

```
| LET LID DOSPT TIPODATO CORIZ CORDER IGUAL CORIZ LEXPR CORDER { $$ = new  
DecVectores($2, $4, this._$.first_line, this._$.first_column, $9); }
```

```
| LET LID DOSPT TIPODATO CORIZ CORDER CORIZ CORDER IGUAL CORIZ  
LEXPCORCHETE CORDER { $$ = new DecVectores($2, $4, this._$.first_line, this._$.first_column,  
$11); }
```

,

ASIGNACION

```
: ID IGUAL EXP { $$ = new Asignacion($1, $3, this._$.first_line, this._$.first_column); }
| ID CORIZ EXP CORDER IGUAL EXP { $$ = new ModiVectores($1, $3, null, $6, this._$.first_line,
this._$.first_column); }
| ID CORIZ EXP CORDER CORIZ EXP CORDER IGUAL EXP { $$ = new ModiVectores($1, $3,
$6, $9, this._$.first_line, this._$.first_column); }
;
```

IMPRIMIR

```
: PRINT EXP { $$ = new Imprimir($2, this._$.first_line, this._$.first_column); }
;
```

FIF

```
: IF PARIZ EXP PARDER LLAVEIZ LINS LLAVEDER { $$ = new If($3, $6, undefined, undefined,
this._$.first_line, this._$.first_column); }
| IF PARIZ EXP PARDER LLAVEIZ LLAVEDER { $$ = new If($3, undefined, undefined,
undefined, this._$.first_line, this._$.first_column); }
| IF PARIZ EXP PARDER LLAVEIZ LINS LLAVEDER ELSE FIF { $$ = new If($3, $6, undefined,
$9, this._$.first_line, this._$.first_column); }
| IF PARIZ EXP PARDER LLAVEIZ LINS LLAVEDER ELSE LLAVEIZ LINS LLAVEDER { $$ =
new If($3, $6, $10, undefined, this._$.first_line, this._$.first_column); }
| IF PARIZ EXP PARDER LLAVEIZ LLAVEDER ELSE LLAVEIZ LLAVEDER { $$ = new If($3,
undefined, undefined, undefined, this._$.first_line, this._$.first_column); }
;
```

FWHILE

```
: WHILE PARIZ EXP PARDER LLAVEIZ LINS LLAVEDER { $$ = new While($3, $6,
this._$.first_line, this._$.first_column); }
| WHILE PARIZ EXP PARDER LLAVEIZ LLAVEDER { $$ = new While($3, undefined,
this._$.first_line, this._$.first_column); }
;
```

FSWITCH

```
: SWITCH PARIZ EXP PARDER LLAVEIZ LCASES DEFAULT DOSPT LINS LLAVEDER { $$ =
new Switch($3, $6, $9, this._$.first_line, this._$.first_column); }
| SWITCH PARIZ EXP PARDER LLAVEIZ DEFAULT DOSPT LINS LLAVEDER { $$ = new
Switch($3, undefined, $8, this._$.first_line, this._$.first_column); }
| SWITCH PARIZ EXP PARDER LLAVEIZ LCASES LLAVEDER { $$ = new Switch($3, $6,
undefined, this._$.first_line, this._$.first_column); }
| SWITCH PARIZ EXP PARDER LLAVEIZ LLAVEDER { $$ = new Switch($3, undefined,
undefined, this._$.first_line, this._$.first_column); }
;
```

LCASES

```
: LCASES CASE EXP DOSPT LINS { $$ = $1; $$.$push({case: $3, INS: $5}); }
| CASE EXP DOSPT LINS { $$ = [{case: $2, INS: $4}]; }
;
```

```

FBREAK
: BREAK PTCOMA { $$ = new Break(this._$.first_line, this._$.first_column); }
;

FCONTINUE
: CONTINUE PTCOMA { $$ = new Continuar(this._$.first_line, this._$.first_column); }
;

FFOR
:FOR PARIZ DECLARACION PTCOMA EXP PTCOMA ACTUALIZACION LLAVEIZ LINS
LLAVEDER { $$ = new For($3, $5, $7, $9, this._$.first_line, this._$.first_column); }
|FOR PARIZ ASIGNACION PTCOMA EXP PTCOMA ACTUALIZACION LLAVEIZ LLAVEDER {
$$ = new For($3, $5, $7, $9, this._$.first_line, this._$.first_column); }

;

ACTUALIZACION
:ASIGNACION PARDER { $$ = $1; }
|INCREMENTO PARDER { $$ = $1; }
|DECREMENTO PARDER{ $$ = $1; }
;

FLOOP
:LOOP LLAVEIZ LINS LLAVEDER { $$ = new Loop($3, this._$.first_line, this._$.first_column); }
;

DOWHILE
: DO LLAVEIZ LINS LLAVEDER UNTIL PARIZ EXP PARDER { $$ = new DoWhile($3, $7,
this._$.first_line, this._$.first_column); }
| DO LLAVEIZ LLAVEDER UNTIL PARIZ EXP PARDER { $$ = new DoWhile(undefined, $7,
this._$.first_line, this._$.first_column); }
;

FRETURN
: RETURN EXP PTCOMA { $$ = new Return($2, this._$.first_line, this._$.first_column); }
| RETURN PTCOMA { $$ = new Return(undefined, this._$.first_line, this._$.first_column); }
;

LLAMADA
: ID PARIZ PARAMETROS_2 PARDER { $$ = new Call($1, $3, this._$.first_line,
this._$.first_column); }
| ID PARIZ PARDER { $$ = new Call($1, [], this._$.first_line, this._$.first_column); }
;

FFUNCION
: FUNCTION TIPODATO ID PARIZ PARAMETROS PARDER LLAVEIZ LINS LLAVEDER { $$ =
new Funcion($2, $3, $5, $8, this._$.first_line, this._$.first_column); }
| FUNCTION VOID ID PARIZ PARAMETROS PARDER LLAVEIZ LINS LLAVEDER { $$ = new
Funcion("Void", $3, $5, $8, this._$.first_line, this._$.first_column); }
| FUNCTION TIPODATO ID PARIZ PARDER LLAVEIZ LINS LLAVEDER { $$ = new Funcion($2,
$3, [], $7, this._$.first_line, this._$.first_column); }
| FUNCTION VOID ID PARIZ PARDER LLAVEIZ LINS LLAVEDER { $$ = new Funcion("Void",
$3, [], $7, this._$.first_line, this._$.first_column); }

```

PARAMETROS

```
:PARAMETROS COMA PARAMETRO { $$ = $1; $$push($3); }  
|PARAMETRO { $$ = [$1]; }  
;
```

PARAMETRO

```
:ID DOSPT TIPODATO IGUAL EXP { $$ = {id: $1, tipo: $3, valor: $4}; }  
|ID DOSPT TIPODATO { $$ = {id: $1, tipo: $3, valor: undefined}; }  
;
```

PARAMETROS_2

```
:PARAMETROS_2 COMA PARAMRO_2 { $$ = $1; $$push($3); }  
|PARAMRO_2 { $$ = [$1]; }  
;
```

PARAMRO_2

```
:ID IGUAL EXP { $$ = {id: $1, valor: $3}; }  
;
```

EXP

```
: EXP MAS EXP      {$$ = new Arithmetica($1, "+", $3, this._$.first_line, this._$.first_column);}  
| EXP MENOS EXP    {$$ = new Arithmetica($1, "-", $3, this._$.first_line, this._$.first_column);}  
| EXP POR EXP      {$$ = new Arithmetica($1, "*", $3, this._$.first_line, this._$.first_column);}  
| EXP DIV EXP      {$$ = new Arithmetica($1, "/", $3, this._$.first_line, this._$.first_column);}  
| EXP MOD EXP      {$$ = new Arithmetica($1, "%", $3, this._$.first_line, this._$.first_column);}  
| EXP ELEV EXP      {$$ = new Arithmetica($1, "^", $3, this._$.first_line, this._$.first_column);}  
| EXP RAIZ EXP      {$$ = new Arithmetica($1, "$", $3, this._$.first_line, this._$.first_column);}  
| EXP AND EXP      {$$ = new OpLogicos($1, $3, "&&", this._$.first_line, this._$.first_column);}  
| EXP OR EXP       {$$ = new OpLogicos($1, $3, "||", this._$.first_line, this._$.first_column);}  
| NOT EXP          {$$ = new OpLogicos(null, $2, "!", this._$.first_line, this._$.first_column);}  
| PARDER EXP PARDER {$$ = $2;}  
| EXP MENOR EXP    {$$ = new OpRelacionales($1, $3, "<", this._$.first_line,  
this._$.first_column);}  
| EXP MAYOR EXP    {$$ = new OpRelacionales($1, $3, ">", $3, this._$.first_line,  
this._$.first_column);}  
| EXP MENORIGUAL EXP {$$ = new OpRelacionales($1, $3, "<=", $3, this._$.first_line,  
this._$.first_column);}  
| EXP MAYORIGUAL EXP {$$ = new OpRelacionales($1, $3, ">=", $3, this._$.first_line,  
this._$.first_column);}  
| EXP DIFERENTE EXP {$$ = new OpRelacionales($1, $3, "!=", $3, this._$.first_line,  
this._$.first_column);}  
| EXP IIGUAL EXP   {$$ = new OpRelacionales($1, $3, "==", $3, this._$.first_line,  
this._$.first_column);}  
| DTERNARIO        {$$ = $1;}  
| CASTEOS           {$$ = $1;}  
| INCREMENTO        {$$ = $1;}  
| DECREMENTO        {$$ = $1;}  
| F                 {$$ = $1;}  
| ID CORIZ EXP CORDER {$$ = new AccesoVector($1, $3, null, this._$.first_line,  
this._$.first_column), }
```

```

|ID CORIZ EXP CORDER CORIZ EXP CORDER { $$ = new AccesoVector($1, $3, $6,
this._$.first_line, this._$.first_column); }
|LLAMADA      { $$ = $1; }
|PARIZ EXP PARDER { $$ = $2; }
|FNATIVAS     { $$ = $1; }
| MENOS EXP %prec UMENOS{ $$ = new NegacionUnaria("-", $2, this._$.first_line,
this._$.first_column); }
;

```

```

TIPODATO
: INT      { $$ = TipoDato.ENTERO; }
| DOUBLE   { $$ = TipoDato.DOUBLE; }
| BOOLEAN  { $$ = TipoDato.BOOLEANO; }
| CHAR     { $$ = TipoDato.CHAR; }
| STRING   { $$ = TipoDato.CADENA; }
;

```

```

LEXPR
: LEXPR COMA EXP { $$ = $1; $.push($3); }
| EXP           { $$ = [$1]; }
;

```

```

LID
: LID COMA ID { $$ = $1; $.push($3); }
| ID         { $$ = [$1]; }
;

```

```

LEXPCORCHETE
: LEXPCORCHETE COMA CORIZ LEXPR CORDER { $$ = $1; $.push($4); }
| CORIZ LEXPR CORDER { $$ = [$2]; }
;

```

```

DTERNARIO
: IF PARIZ EXP PARDER EXP DOSPT EXP { $$ = new Ternario($3, $5, $7, this._$.first_line,
this._$.first_column); }
;

```

```

CASTEOS
: CAST PARIZ EXP AS TIPODATO PARDER %prec FCAST { $$ = new Cast($5, $3,
this._$.first_line, this._$.first_column); }
;

```

```

INCREMENTO
: EXP PLUS { $$ = new IncrementoDecremento($1, "++", this._$.first_line, this._$.first_column); }
;

```

```

DECREMENTO
: EXP MIN { $$ = new IncrementoDecremento($1, "--", this._$.first_line, this._$.first_column); }
;

```

```

F
.DOUBLE{ $$ = new Literal($1, TipoDato.DOUBLE, @1.first_line, @1.first_column); }

```

```
|ENTERO{ $$ = new Literal($1, TipoDato.ENTERO, @1.first_line, @1.first_column); }  
|TRUE{ $$ = new Literal('true', TipoDato.BOOLEANO, @1.first_line, @1.first_column); }  
|FALSE{ $$ = new Literal('false', TipoDato.BOOLEANO, @1.first_line, @1.first_column); }  
|CARACTER{ $$ = new Literal($1, TipoDato.CHAR, @1.first_line, @1.first_column); }  
|Cadena{ $$ = new Literal($1, TipoDato.CADENA, @1.first_line, @1.first_column); }  
|ID {  $$ = new Acceso($1, @1.first_line, @1.first_column); }  
;
```

FNATIVAS

```
:LOWER PARIZ EXP PARDER { $$ = new Nativas("LOWER", $3, this._$.first_line,
this._$.first_column); }
|UPPER PARIZ EXP PARDER { $$ = new Nativas("UPPER", $3, this._$.first_line,
this._$.first_column); }
|LENGTH PARIZ ID PARDER { $$ = new Len("LEN",$3, this._$.first_line, this._$.first_column); }
|TRUNCATE PARIZ EXP PARDER { $$ = new Nativas("TRUNCATE", $3, this._$.first_line,
this._$.first_column); }
|ROUND PARIZ EXP PARDER { $$ = new Nativas("ROUND", $3, this._$.first_line,
this._$.first_column); }
|EXP IS TIPODATO { $$ = new FunIs($3, $1, this._$.first_line, this._$.first_column); }
|TOSTRING PARIZ EXP PARDER { $$ = new FuncionNativa("TOSTRING", $3, this._$.first_line,
this._$.first_column); }
|CHARARRAY PARIZ EXP PARDER { $$ = new FuncionNativa("TOCHARARRAY", $3,
this._$.first_line, this._$.first_column); }
|REVERSE PARIZ ID PARDER { $$ = new Len("REVERSE",$3, this._$.first_line,
this._$.first_column); }
|MAX PARIZ ID PARDER { $$ = new Len("MAX",$3, this._$.first_line, this._$.first_column); }
|MIN2 PARIZ ID PARDER { $$ = new Len("MIN",$3, this._$.first_line, this._$.first_column); }
|SUM PARIZ ID PARDER { $$ = new Len("SUM",$3, this._$.first_line, this._$.first_column); }
|AVERAGE PARIZ ID PARDER { $$ = new Len("AVERAGE",$3, this._$.first_line,
this._$.first_column); }
```

;